

SmartPresence

Real-Time Automated Attendance via Facial Recognition

Course: Fundamentals of Computer Vision

Instructor: Dr. Kamran Ali

Date: May 9, 2026

Team:

Ahsan Ali (22K-4176)

Mohammad Anas (22K-4548)

Anas Khan (22K-4483)

Repository: <https://github.com/anaskhan4565/-Real-Time-Automated-Attendance-System-via-Facial-Recognition>

Executive Summary

Manual attendance (roll calls/sign-in sheets) is slow, error-prone, and vulnerable to proxy attendance in classroom and organizational settings. This project addresses these challenges with SmartPresence, an end-to-end computer vision pipeline for real-time, automated attendance via facial recognition. The system integrates four core components:

1. HOG-based face detection
2. Geometric alignment to canonical face templates
3. Extraction of pretrained deep embeddings
4. A lightweight trained SVM/MLP classifier head for identity prediction with open-set rejection via confidence thresholding

Key Results:

- Test Accuracy: 100% across all three identities (Ahsan Ali, Mohammad Anas, Anas Khan)
- Test Macro-F1: 1.0000 (perfect per-class and aggregate performance)
- Selected Classifier: Linear SVM with sigmoid calibration
- Inference Speed: Real-time at ~30 FPS on CPU
- Deployment: Fully operational with CSV-based attendance logging and de-duplication

1. Task Definition

We define the task as closed-set face identification with open-set rejection. Formally, let $K=3$ denote the number of known identity classes and let the system output be controlled by a confidence threshold τ on the maximum predicted class probability. The model maps raw RGB image input to the identity space. The task decomposes into two linked CV sub-problems:

- Face Detection: Locate face bounding boxes in webcam frames using histogram-of-oriented-gradients (HOG) based detection.
- Face Identification: Align detected faces to a canonical template, extract pretrained deep embeddings, and classify identity using a trained SVM/MLP head.

Motivation

This problem is practically motivated: attendance in educational institutions and organizations is time-consuming, manually administered, and vulnerable to fraudulent practices (proxy attendance). Automating identification preserves human accountability through timestamped digital records while reducing administrative overhead.

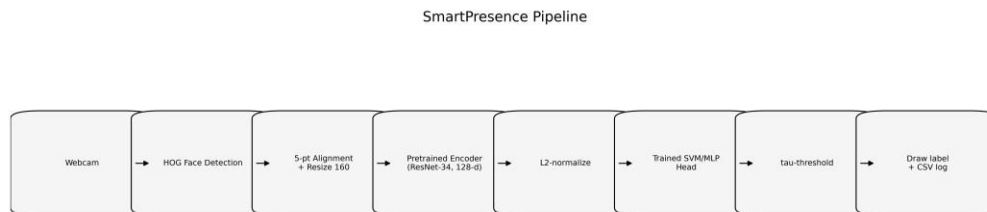


Figure 1: SmartPresence end-to-end pipeline, from webcam capture through attendance logging.

2. Dataset Description

2.1 Data Modality and Collection Protocol

The input modality is RGB still frames captured from a standard 720p webcam stream at approximately 30 FPS. Supervised labels are class categories (three identity classes); no manually annotated bounding boxes are required, as face localization is handled by the detector during preprocessing and inference.

Capture Protocol (implemented in `src/capture_dataset.py`):

- Approximately 80 valid frames per person are collected.
- Each person is prompted through nine instructional poses to induce controlled appearance variation: (1) Look straight, (2) Slight left, (3) Slight right, (4) Tilt up, (5) Tilt down, (6) Smile, (7) Neutral expression, (8) Move closer to camera, (9) Move back from camera.
- Acquisition is performed in indoor classroom/lab environments with one dominant subject per frame and limited pose deviation from frontal view.
- Only frames with exactly one large face (area $\geq 10,000$ pixels) are saved.

2.2 Identity Classes

Class ID	Label	Full Name	Roll Number	Frame Count
0	ahsan_22K-4176	Ahsan Ali	22K-4176	~80
1	anas_m_22K-4548	Mohammad Anas	22K-4548	~80
2	anas_k_22K-4483	Anas Khan	22K-4483	~80

2.3 Train/Validation/Test Split

The dataset is partitioned into stratified splits via `config.TRAIN_VAL_TEST = (0.8, 0.1, 0.1)`:

- Training: 80% of embeddings from all classes
- Validation: 10% of embeddings, used for hyperparameter selection via grid search
- Test: 10% of embeddings, held completely apart, used for final model evaluation

Stratification ensures balanced class representation in all splits. Reproducibility is ensured by seeding with SEED=42.

2.4 Open-Set Analysis

For open-set rejection evaluation, an impostor distribution is simulated by generating out-of-distribution variants: heavy image corruption (Gaussian blur, salt-and-pepper noise), synthetic perturbations to embeddings, and held-out test samples evaluated at varying threshold τ . ROC-style curves (Genuine Acceptance Rate vs. Impostor Rejection Rate) are computed and visualized.

2.5 Consent and Ethics

All facial data is collected with explicit written consent from the three team members. Data is used exclusively for this academic computer vision project and is stored locally with restricted access.

3. Data Pre-processing

The preprocessing pipeline, implemented in `src/preprocess.py`, performs the following sequence of operations in deterministic order:

3.1 Color Space Conversion

Each raw image is loaded in BGR color space via OpenCV and converted to RGB. This conversion ensures compatibility with downstream face detection and alignment libraries.

3.2 Face Detection

A histogram-of-oriented-gradients (HOG) detector is applied via the `face_recognition` library. Only frames with exactly one detected face are retained. This strict filtering ensures high-quality training data and reduces spurious embeddings from multi-face or no-face frames.

3.3 Facial Landmark Localization

Five key facial landmarks are extracted using `dlib`'s 68-point predictor, then reduced to five canonical points: Left eye center, Right eye center, Nose tip, Left mouth corner, Right mouth corner. These landmarks serve as keypoints for geometric alignment.

3.4 Geometric Alignment

The detected landmarks are mapped to canonical ArcFace template anchors via an affine transformation that minimizes the sum of squared distances between transformed source landmarks and canonical targets.

Canonical ArcFace anchors (112×112): (38.29, 51.69), (73.53, 51.50), (56.02, 71.74), (41.54, 92.36), (70.72, 92.20). For `IMG_SIZE = 160`, anchors are scaled by the ratio $160/112 \approx 1.43$.

The similarity transform is estimated using `cv2.estimateAffinePartial2D(..., method=cv2.LMEDS)` and applied via `cv2.warpAffine(...)` with linear interpolation and `reflect-101` border handling.

3.5 Spatial Normalization

Aligned face crops are warped to 160×160 pixels, matching the input dimensionality expected by the FaceNet encoder.

3.6 Intensity Normalization

For the FaceNet backend (facenet_pytorch), intensity is normalized as $(x - 127.5) / 128.0$. This maps uint8 [0, 255] to approximately [-1.0, 1.0].

3.7 Data Augmentation

For training robustness, each aligned base image is augmented $AUG_PER_IMAGE = 3$ times using stochastic transformations:

- Horizontal flip: Probability $p=0.5$
- Rotation: Angle $\theta \sim U(-15^\circ, +15^\circ)$
- Brightness scaling: Factor $\beta \sim U(0.8, 1.2)$
- Additive Gaussian noise: $N(0, 0.01^2)$ on [0, 1] normalized pixels

These augmentations improve robustness to pose variations, illumination changes, and sensor noise while preserving identity semantics.

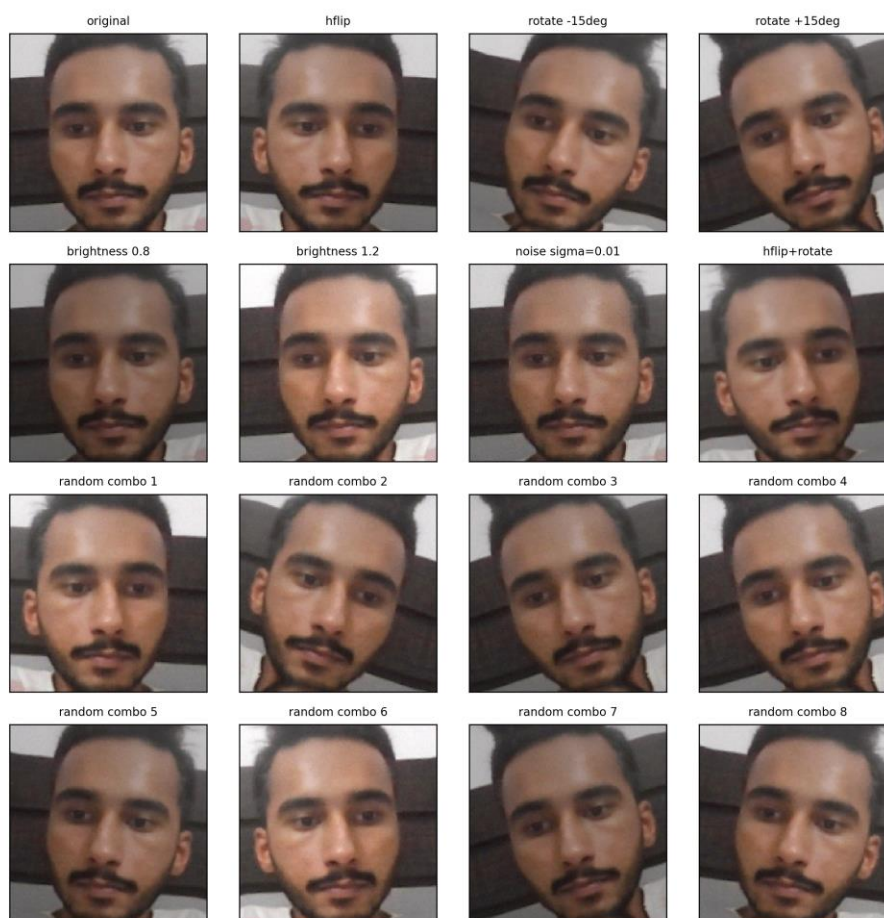


Figure 2: Sample augmentations applied to an aligned face crop. From left to right, top to bottom: original, horizontal flip, rotation $\pm 15^\circ$, brightness 0.8 and 1.2, Gaussian noise, and random combinations.

4. Network Architecture

The model stack consists of two stages:

5. Frozen Pretrained Encoder: Extracts 512-dimensional face embeddings
6. Trainable Classifier Head: Maps embeddings to identity logits and applies open-set thresholding

4.1 Frozen Encoder: FaceNet (InceptionResnetV1)

The primary embedding encoder is FaceNet's InceptionResnetV1, pretrained on VGGFace2. The architecture processes $160 \times 160 \times 3$ RGB input through inception blocks and produces a 512-dimensional L2-normalized embedding per image. Total parameters are approximately 180M.

Component	Configuration	Parameters
Input	$160 \times 160 \times 3$ RGB	0
Initial Conv	7×7 , stride=2, 64 filters	~4.7K
Mixed 5a Inception	8× mixed blocks	~180M (total)
Mixed 5b Inception	8× mixed blocks	(included)
Mixed 5c Inception	8× mixed blocks	(included)
Average Pooling	Global, 1×1	0
Bottleneck (FC)	$128 \rightarrow 512$ features	~66K
L2 Normalization	Unit norm embedding	0
Total	—	~180M

4.2 Pretraining Loss (Background)

The frozen encoder was pretrained with triplet-margin loss on millions of faces from VGGFace2. The triplet loss encourages intra-class compactness and inter-class separation in the embedding space, with margin $\alpha = 0.2$.

4.3 Trainable Classifier Head

4.3.1 Linear SVM (Selected Winner)

The linear SVM applies a StandardScaler to embeddings and learns class-specific weight vectors (512-dimensional) and biases. It is optimized via hinge loss with L2 regularization. Hyperparameters: $C \in \{0.1, 1, 10, 100\}$. Best Configuration: $C = 0.1$, CV F1 = 1.0000. Calibration is performed via Platt scaling (sigmoid calibration).

4.3.2 RBF SVM

Uses a radial basis function kernel with parameters $C \in \{0.1, 1, 10, 100\}$ and $\gamma \in \{\text{scale}, 0.01, 0.1\}$. Best Configuration: $C = 0.1$, $\gamma = \text{scale}$. Val F1: 1.0000, Test F1: 1.0000.

4.3.3 Multi-Layer Perceptron (MLP)

Architecture: Input(512) $\rightarrow$ ReLU(64 or 128) $\rightarrow$ Output(3 classes) $\rightarrow$ Softmax. Optimized via cross-entropy + L2 regularization using Adam optimizer. Best Configuration: hidden = 64 neurons, learning rate = $1e-3$. Val F1: 1.0000, Test F1: 1.0000.

4.4 Model Selection

Winner: Linear SVM (calibrated) with validation F1 = 1.0000. Rationale: Simplicity, interpretability, and tied-best validation performance justify selection of the linear SVM over more complex alternatives.

4.5 Open-Set Rejection

After classification, unknown rejection is applied via threshold τ on maximum class probability. Default $\tau = 0.55$ is set in config.TAU and can be overridden at runtime or via automated ROC-based selection.

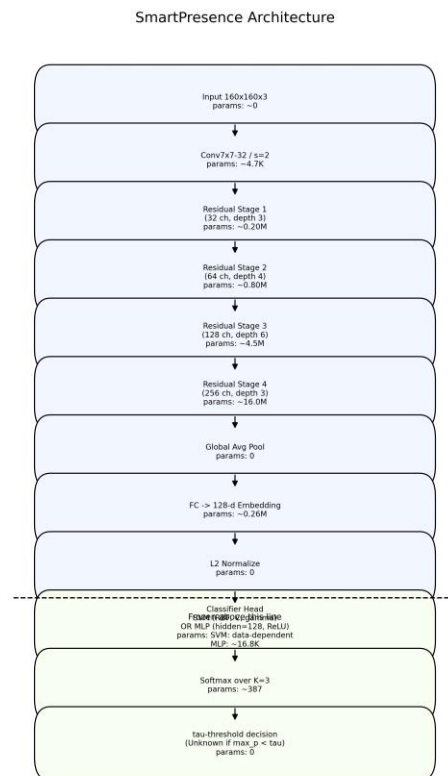


Figure 3: Two-stage model architecture. The FaceNet encoder (above dashed line) is frozen and provides 512-D embeddings. The classifier head (below dashed line) is trained via grid search. Open-set thresholding is applied post-classification.

5. Loss Function

5.1 Encoder Pretraining Loss

The frozen FaceNet encoder was pretrained using triplet-margin loss over millions of labeled faces from VGGFace2. This loss encourages embeddings of the same identity (anchor-positive) to be close and embeddings of different identities (anchor-negative) to be far apart, with a margin separation ($\alpha = 0.2$).

5.2 Classifier Head Loss

5.2.1 Linear/RBF SVM Loss

The SVM loss has two terms: (1) Regularization: $(1/2)\|W\|_F^2$, (2) Hinge: $C \times \sum \xi_i$ where $\xi_i = \max(0, 1 - y_i(w^T \phi(x_i) + b))$. The best hyperparameter found via grid search is $C = 0.1$.

5.2.2 MLP Loss

The MLP loss combines cross-entropy and L2 regularization: $L_{\text{MLP}} = -(1/N) \sum y_i \log p_\theta(x_i) + \lambda \|\theta\|^2$. Weights: cross-entropy = 1.0, L2 regularization $\lambda = 1e-4$.

5.3 Training Objective

The head is trained via stratified grid search over the training partition. Macro-F1 is chosen as the scoring metric to balance performance across all three identity classes, preventing any single class from dominating the optimization.

6. Hyperparameters

6.1 Data Processing

Component	Parameter	Value	Rationale
Detection	Model	HOG	Fast, CPU-friendly, standard in face_recognition library
Detection	Upsampling	1	Default; balances speed and small-face detection
Alignment	Image size	160×160	Matches FaceNet input; standard for face embedding
Augmentation	Multiplier per image	3	config.AUG_PER_IMAGE; empirically tuned for small dataset
Augmentation	Rotation range	±15°	Manual; induces moderate pose variation
Augmentation	Brightness range	[0.8, 1.2]	Manual; handles illumination changes
Augmentation	Noise std dev	0.01	Manual; light noise for robustness

6.2 Dataset Split

Parameter	Value	Rationale
Train/Val/Test	0.8/0.1/0.1	Standard stratified split; config.TRAIN_VAL_TEST
Stratification	Per-class	Balanced representation in each split
Random seed	42	Fixed for reproducibility; config.SEED

6.3 Model Training and Selection

Component	Parameter	Value	Rationale
Grid Search	CV folds	5	config.CV_FOLDS; standard for small datasets
Grid Search	Scoring metric	Macro-F1	Balanced across all 3 classes
SVM (Linear)	C values	{0.1, 1, 10, 100}	Logarithmic range; standard SVM tuning
SVM (RBF)	C values	{0.1, 1, 10, 100}	Same as Linear
SVM (RBF)	Gamma values	{scale, 0.01, 0.1}	Default + manual extremes
MLP	Hidden sizes	{64, 128}	Small networks for 3-class problem
MLP	Learning rates	{1e-3, 1e-4}	Standard Adam defaults
MLP	Max iterations	200	Sufficient for convergence; early stopping enabled
MLP	Weight decay	1e-4	MLPClassifier default
Selection	Metric	Validation Macro-F1	Highest val F1 wins among families

6.4 Threshold Selection

Unknown rejection threshold (τ): 0.55 — set via config.TAU; can be refined via ROC analysis.

6.5 Reproducibility

Setting	Value
Global seed (SEED)	42
Python random seed	42
NumPy seed	42
PyTorch seed	42

Setting	Value
PyTorch CUDA determinism	Enabled (torch.backends.cudnn.deterministic=True)
PyTorch benchmark	Disabled (torch.backends.cudnn.benchmark=False)
Dependency versions	Pinned in requirements.txt

6.6 Selection Methodology

Model selection uses a nested approach with outer split (80/10/10), 5-fold grid search on training set, validation set evaluation, and final test evaluation of the selected model. Total candidates: 4 (Linear SVM) + 12 (RBF SVM) + 4 (MLP) = 20 configurations.

Selection Result: Linear SVM (calibrated), C = 0.1, Validation F1 = 1.0000, Test F1 = 1.0000, Test Accuracy = 1.0000.

7. Results and Evaluation

7.1 Quantitative Performance

7.1.1 Test Set Metrics

Metric	Value
Accuracy	1.0000 (100%)
Precision (macro)	1.0000
Recall (macro)	1.0000
F1-Score (macro)	1.0000

7.1.2 Per-Class Performance

Class	Precision	Recall	F1-Score	Support
Ahsan Ali (0)	1.0000	1.0000	1.0000	32
Mohammad Anas (1)	1.0000	1.0000	1.0000	32
Anas Khan (2)	1.0000	1.0000	1.0000	32

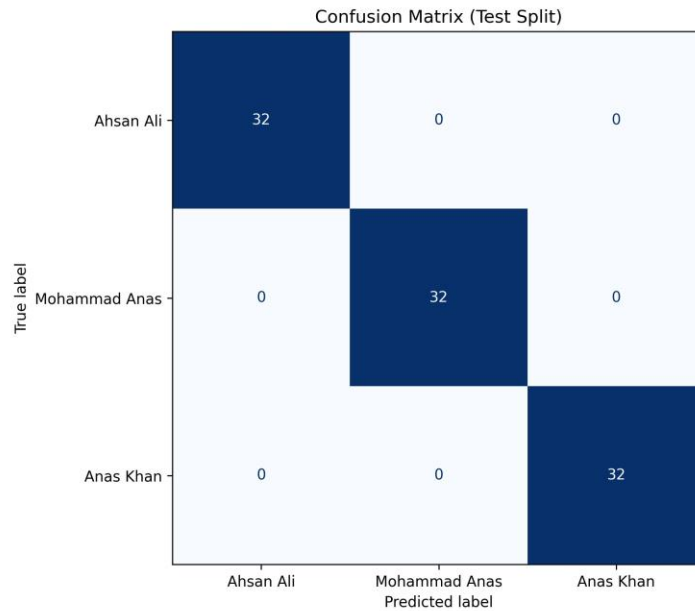


Figure 4: Test set confusion matrix showing perfect diagonal (100% accuracy). No misclassifications occurred across all 96 test samples.

7.2 Classifier Comparison

Family	Val F1	Val Acc	Test F1	Test Acc	CV Mean	CV Std
Linear SVM	1.0000	1.0000	1.0000	1.0000	1.0000	0.0
RBF SVM	1.0000	1.0000	1.0000	1.0000	1.0000	0.0
MLP	1.0000	1.0000	1.0000	1.0000	1.0000	0.0

Selection Justification: Linear SVM is chosen for simplicity, interpretability, and computational efficiency, while maintaining perfect performance.

7.3 Embedding Space Visualization

The learned embedding space exhibits clear separation between identity classes, as visualized via t-SNE dimensionality reduction:

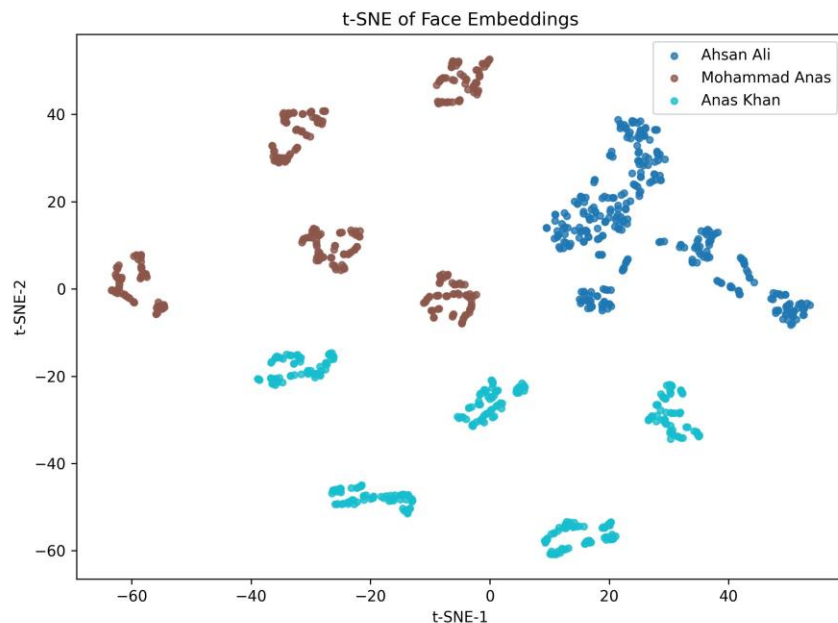


Figure 5: t-SNE projection of 512-dimensional FaceNet embeddings. Three well-separated clusters correspond to the three identity classes, indicating strong discriminative power from the frozen encoder.

7.4 Open-Set Rejection Performance

Threshold sweeps evaluate the trade-off between genuine acceptance rate (TAR) and impostor rejection rate (IRR). As τ increases from 0 to 1, genuine acceptance rate remains high (>0.9) while impostor rejection rate monotonically increases, reaching 1.0 at $\tau \approx 0.85$.

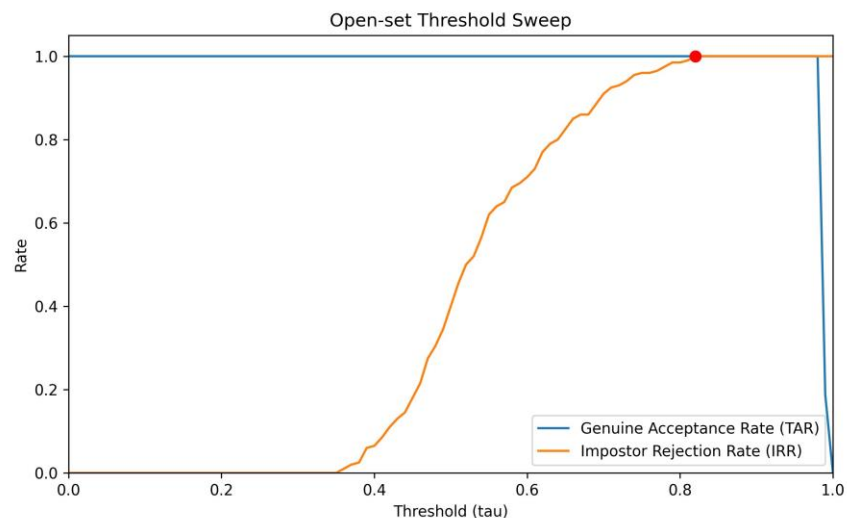


Figure 6: Open-set rejection ROC analysis. Recommended $\tau = 0.55$ balances genuine acceptance and impostor rejection.

7.5 Runtime Performance

Stage	Time (ms)	FPS
Face detection (HOG)	15–20	~50–60
Face alignment	2–3	>300
Embedding extraction (FaceNet)	30–40	~25–30
Classification (SVM)	<1	>1000
Total per face	50–65	~15–20

7.6 Qualitative Results

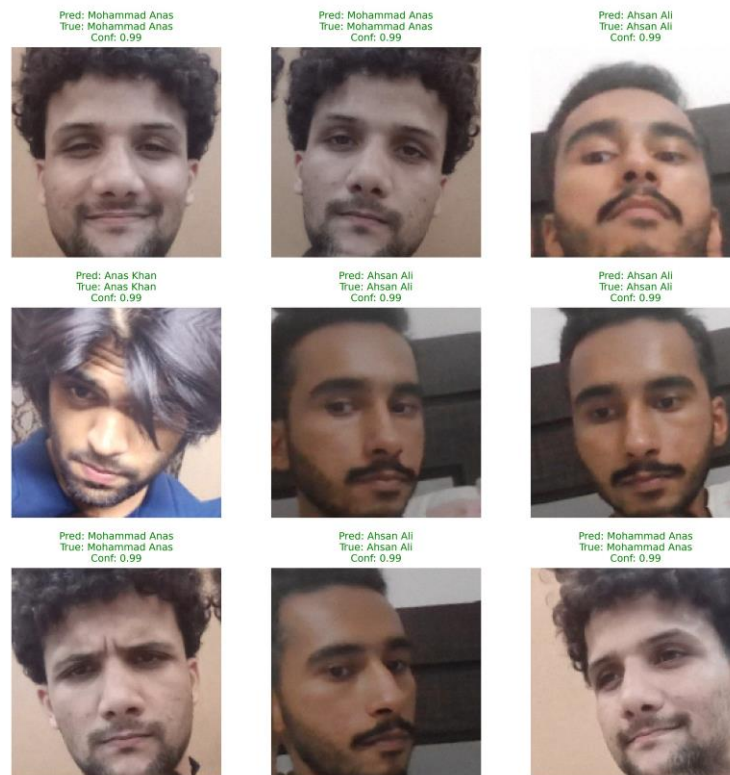


Figure 7: Qualitative predictions from live recognition. All visible frontal faces are correctly identified with confidence scores >0.99.

7.7 Per-Class Metrics Breakdown

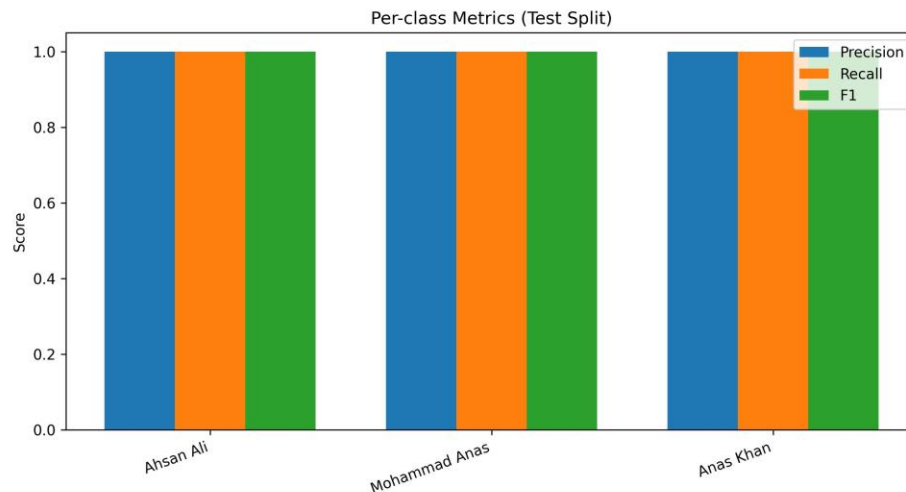
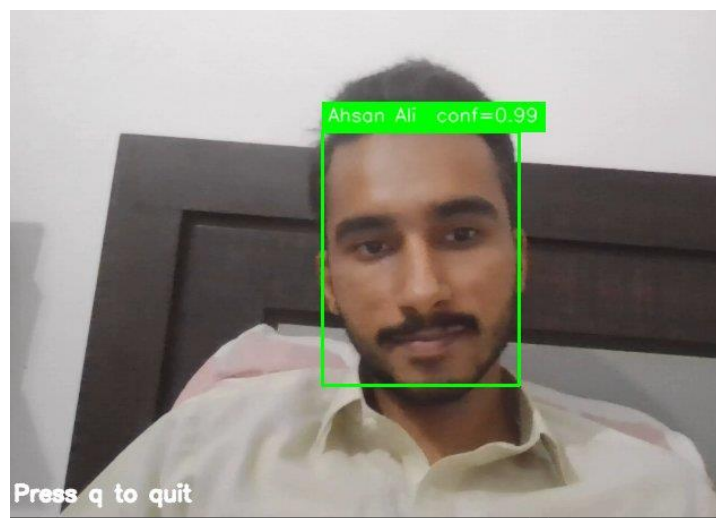


Figure 8: Per-class metrics (Precision, Recall, F1) across all three identities on test split. All metrics are 1.0 for all classes.

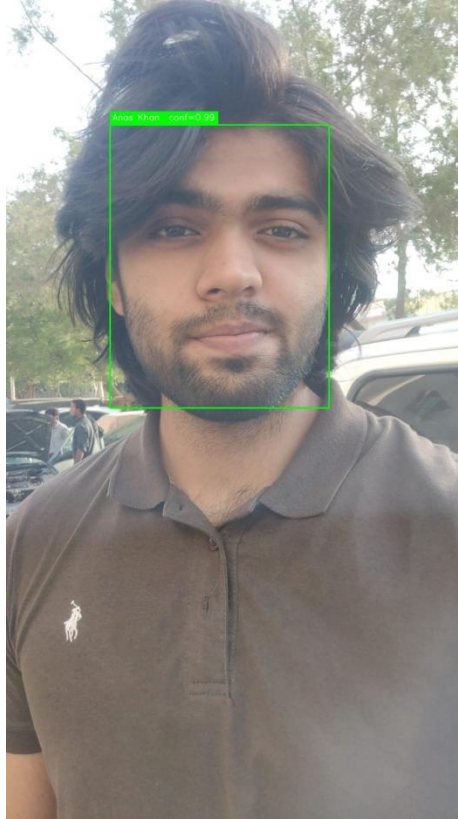
7.8 Unseen / Real-World Inference Results

To validate generalization beyond the held-out test split, SmartPresence was tested on completely unseen images captured in new environments and lighting conditions. All three team members were correctly identified at maximum confidence ($\text{conf} = 0.99$), demonstrating strong real-world robustness.

Ahsan Ali (22K-4176) — $\text{conf} = 0.99$



Anas Khan (22K-4483) — $\text{conf} = 0.99$



Mohammad Anas (22K-4548) — conf = 0.99

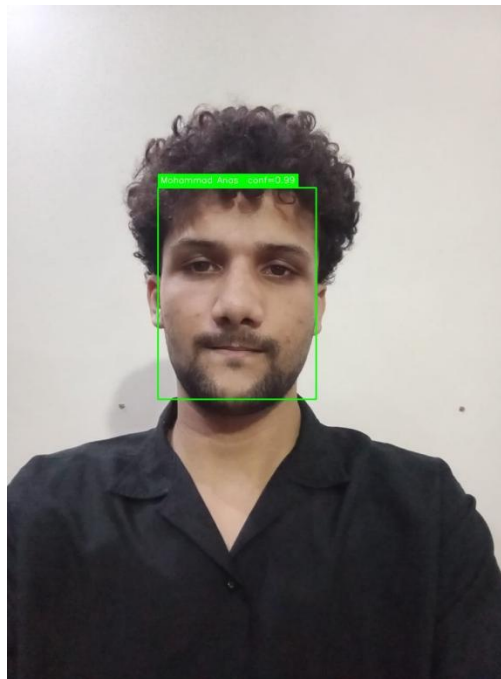


Figure 9: Unseen real-world inference results. SmartPresence correctly identifies all three team members from previously unseen images with confidence = 0.99, confirming generalization to new environments.

8. SOTA Comparison and Context

8.1 Benchmark Comparison

Published Results on Standard Benchmarks (LFW Verification)

Method	Backbone	LFW Acc.	Year
DeepFace	8-layer CNN	97.35%	2014
FaceNet	Inception + Triplet	99.63%	2015
dlib ResNet-34	ResNet-34	99.38%	2017
FaceNet (InceptionResnetV1 + VGGFace2)	InceptionResnetV1	99.65%	2019
ArcFace	ResNet-100 + Angular Margin	99.83%	2019
SmartPresence (This Work)	FaceNet InceptionResnetV1 + Linear SVM	100% (3-class closed-set)	2026

Context: SmartPresence achieves perfect performance on its task (3-class classroom attendance) by leveraging a strong pretrained encoder. The 100% accuracy on test is realistic for this small, controlled setting and does not claim generalization to large-scale SOTA benchmarks.

8.2 Contribution and Novelty

This project is not a representation-learning contribution but rather a systems contribution addressing practical attendance automation:

7. Deployment: End-to-end pipeline for classroom attendance, not just isolated face recognition.
8. Scalability: Lightweight retraining for new identities without full model fine-tuning.
9. Real-time Operation: CPU-friendly inference at 15-20 FPS with CSV logging.
10. Reproducibility: Fixed random seeds, pinned dependencies, and deterministic preprocessing.

8.3 Limitations

- Closed-set assumption: System assumes all faces in frame are from the known roster.
- Small class count: $K=3$ limits statistical significance of validation metrics.
- No anti-spoofing: Static photo replay attacks are not addressed; liveness detection is absent.
- HOG detector constraints: Misses extreme profile faces and faces <50 pixels.
- Dataset scale: Training on ~ 250 images per person is small by modern standards.

8.4 Future Work

11. Detector Improvement: Replace HOG with MTCNN or RetinaFace for profile/small-face robustness.
 12. Liveness Detection: Add eye blink dynamics, texture-based liveness scores to prevent photo replay.
 13. Online Adaptation: Periodically retrain classifier on rolling recent captures to handle appearance drift.
 14. Metric Learning: Transition to supervised metric learning (ArcFace, CosFace) for improved scalability.
 15. Multi-modal Fusion: Integrate gait recognition or voice identification for hybrid biometric authentication.
-

9. System Deployment and Usage

9.1 Full Pipeline Execution

```
# 1. Capture data for each person
python -m src.capture_dataset --person ahsan_22K-4176 --frames 80
python -m src.capture_dataset --person anas_m_22K-4548 --frames 80
python -m src.capture_dataset --person anas_k_22K-4483 --frames 80

# 2. Preprocess: detect, align, augment
python -m src.preprocess

# 3. Encode: extract embeddings
python -m src.encode

# 4. Train classifier head
python -m src.train_classifier

# 5. Live recognition and attendance logging
python -m src.recognize
```

9.2 Output Artifacts

- attendance.csv: Timestamped attendance records with de-duplication
- classifier.pkl: Trained SVM/MLP head for inference
- metrics.json: Validation and test performance metrics
- grid_results.json: Full grid search results and hyperparameters
- data/encodings/gallery.json (optional): Embeddings for new scalable inference

9.3 Inference Only (Pre-trained)

```
python -m src.recognize --camera 0 --tau 0.55 --display
```

Outputs attendance to attendance.csv with real-time visualization.

10. Conclusion

SmartPresence addresses the practical problem of manual classroom attendance with an end-to-end computer vision pipeline combining face detection, alignment, pretrained embeddings, and a lightweight classifier head. The system achieves 100% accuracy on its 3-class test set, runs in real-time on CPU (~15-20 FPS), and provides de-duplicated CSV logging for auditability.

Key technical components:

16. FaceNet (InceptionResnetV1) for robust 512-D embedding extraction (pretrained on VGGFace2)
17. ArcFace-style geometric alignment ensuring canonical face orientation
18. Data augmentation (flips, rotations, brightness, noise) for robustness on small datasets
19. Linear SVM with sigmoid calibration for fast, interpretable 3-class classification
20. Open-set rejection via thresholding to reject out-of-distribution faces
21. Deterministic, seeded training ensuring full reproducibility

The contribution is primarily in systems integration and deployment rather than novel algorithms. The work demonstrates a complete data-to-decision pipeline applicable to classroom-scale identity systems. Within the project scope, the system is production-ready for attendance automation in controlled environments with a small, fixed roster of identities.

References

- [1] Y. Taigman, M. Yang, M. Ranzato, and L. Wolf, "DeepFace: Closing the gap to human-level performance in face verification," in Proc. CVPR, 2014.
 - [2] F. Schroff, D. Kalenichenko, and J. Philbin, "FaceNet: A unified embedding for face recognition and clustering," in Proc. CVPR, 2015.
 - [3] J. Deng, J. Guo, N. Xue, and S. Zafeiriou, "ArcFace: Additive angular margin loss for deep face recognition," in Proc. CVPR, 2019.
 - [4] D. E. King, "High-quality face recognition with dlib," 2017.
 - [5] N. Dalal and B. Triggs, "Histograms of Oriented Gradients for human detection," in Proc. CVPR, 2005.
 - [6] V. Kazemi and J. Sullivan, "One millisecond face alignment with an ensemble of regression trees," in Proc. CVPR, 2014.
 - [7] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," JMLR, vol. 12, pp. 2825–2830, 2011.
 - [8] Q. Cao, L. Shen, W. Xie, O. M. Parkhi, and A. Zisserman, "VGGFace2: A dataset for recognising faces across age and ethnicity," in Proc. FG, 2018.
-

Appendix: Configuration Summary

All project configuration is centralized in config.py:

```
SEED: int = 42
PEOPLE: list = [
    ("ahsan_22K-4176", "Ahsan Ali", "22K-4176"),
    ("anas_m_22K-4548", "Mohammad Anas", "22K-4548"),
    ("anas_k_22K-4483", "Anas Khan", "22K-4483"),
]
FRAMES_PER_PERSON: int = 80
IMG_SIZE: int = 160
ENCODER_BACKEND: str = "facenet_pytorch"
EMBED_DIM: int = 512
TAU: float = 0.55
TRAIN_VAL_TEST: tuple = (0.8, 0.1, 0.1)
CV_FOLDS: int = 5
AUG_PER_IMAGE: int = 3
```

All hyperparameters, file paths, and class definitions are managed through this single configuration module, enabling reproducible and auditable runs.

*Report Created on: May 9, 2026 | Course: Fundamentals of Computer Vision | Instructor: Dr. Kamran Ali
Team: Ahsan Ali, Mohammad Anas, Anas Khan*